ECM1413 – Computers and the Internet

Student Name: Cameron Russell

Student ID: 750032575

My Answers to Part 1: The Linux Terminal

Task 1:

```
student@lab000001:~$ pwd
/home/student
student@lab000001:~$ mkdir projects
student@lab000001:~$ ls
 casino1.txt   Documents   ECM1413_Computers_and_the_Internet   Music      pt          test_dir            transcript.txt   Videos
 casino.txt    Downloads   marged_file_backup.txt              Pictures   Public     '\..text'            t.txt
 Desktop       d.txt       merged_file.txt                     projects   Templates   thinclient_drives  '\..txt'
student@lab000001:~$ cd projects/
student@lab000001:~/projects$ pwd
/home/student/projects
student@lab000001:~/projects$ mkdir project1
student@lab000001:~/projects$ mkdir project2
student@lab000001:~/projects$ ls
project1  project2
student@lab000001:~/projects$
```

Task 2:

```
student@lab000001:~/projects$ pwd
/home/student/projects
student@lab000001:~/projects$ cd project1
student@lab000001:~/projects/project1$ pwd
/home/student/projects/project1
student@lab000001:~/projects/project1$ touch empty.txt
student@lab000001:~/projects/project1$ ls
empty.txt
student@lab000001:~/projects/project1$ cd ..
student@lab000001:~/projects$ pwd
/home/student/projects
student@lab000001:~/projects$ cd project2
student@lab000001:~/projects/project2$ pwd
/home/student/projects/project2
student@lab000001:~/projects/project2$ touch empty.txt
student@lab000001:~/projects/project2$ ls
empty.txt
student@lab000001:~/projects/project2$
```

Task 3:

```
student@lab000001:~/projects/project2$ pwd
/home/student/projects/project2
student@lab000001:~/projects/project2$ mv empty.txt empty2.txt
student@lab000001:~/projects/project2$ ls
empty2.txt
student@lab000001:~/projects/project2$ mv empty2.txt ../project1
student@lab000001:~/projects/project2$ cd ..
student@lab000001:~/projects$ pwd
/home/student/projects
student@lab000001:~/projects$ cd project1
student@lab000001:~/projects/project1$ pwd
/home/student/projects/project1
student@lab000001:~/projects/project1$ ls
empty2.txt  empty.txt
student@lab000001:~/projects/project1$
```

Task 4:

```
student@lab000001:~$ cat >> quotes.txt
Hello my name is Cameron!
I am really enjoying linux!
Computer Science is an amazing subject.
I am feeling quite comfortable with the terminal now!!!
student@lab000001:~$ cat quotes.txt
Hello my name is Cameron!
I am really enjoying linux!
Computer Science is an amazing subject.
I am feeling quite comfortable with the terminal now!!!
student@lab000001:~$ ls
 casino1.txt   Documents   ECM1413_Computers_and_the_Internet   Music      pt           Templates   thinclient_drives  '\..txt'
 casino.txt    Downloads   marged_file_backup.txt              Pictures   Public       test_dir    transcript.txt      Videos
 Desktop       d.txt       merged_file.txt                     projects   quotes.txt  '\..text'    t.txt
student@lab000001:~$
```

Task 5:

```
student@lab000001:~$ pwd
/home/student
student@lab000001:~$ cp quotes.txt quotes_copy.txt
student@lab000001:~$ ls
 casino1.txt   Documents   ECM1413_Computers_and_the_Internet   Music      pt              quotes.txt  '\..text'           t.txt
 casino.txt    Downloads   marged_file_backup.txt              Pictures   Public          Templates   thinclient_drives  '\..txt'
 Desktop       d.txt       merged_file.txt                     projects   quotes_copy.txt test_dir    transcript.txt      Videos
student@lab000001:~$ cat quotes_copy.txt
Hello my name is Cameron!
I am really enjoying linux!
Computer Science is an amazing subject.
I am feeling quite comfortable with the terminal now!!!
student@lab000001:~$
```

Task 6:

```
student@lab000001:~$ !! >> quotes.txt
cat quotes_copy.txt  >> quotes.txt
student@lab000001:~$ wc quotes.txt
  8  50 300 quotes.txt
student@lab000001:~$
```

Task 7:

```
student@lab000001:~$ pwd
/home/student
student@lab000001:~$ ls -1a > .hidden.txt
student@lab000001:~$ ls
 casino1.txt   Documents   ECM1413_Computers_and_the_Internet   Music      pt                quotes.txt  '\..text'            t.txt
 casino.txt    Downloads   marged_file_backup.txt              Pictures   Public            Templates    thinclient_drives  '\..txt'
 Desktop       d.txt       merged_file.txt                     projects   quotes_copy.txt   test_dir     transcript.txt      Videos
student@lab000001:~$ cat .hidden.txt
.
..
.bash_history
.bash_logout
.bashrc
.cache
casino1.txt
casino.txt
.cloud-locale-test.skip
.config
Desktop
Documents
Downloads
d.txt
ECM1413_Computers_and_the_Internet
.gnupg
.hidden.txt
.ICEauthority
.local
marged_file_backup.txt
merged_file.txt
Music
.packettracer
.pcsc10
Pictures
.pki
.profile
projects
pt
Public
quotes_copy.txt
quotes.txt
Templates
test_dir
.test.txt
\..text
thinclient_drives
transcript.txt
t.txt
..txt
\..txt
Videos
.Xauthority
.xorgxrdp.10.log
.xorgxrdp.10.log.old
.xsession-errors
student@lab000001:~$
```

Task 8:

```
student@lab000001:~$ pwd
/home/student
student@lab000001:~$ man chmod
student@lab000001:~$ man chmod
student@lab000001:~$ chmod u=rw-,g=r--,o=--- quotes.txt
student@lab000001:~$ ls -l
total 764
-rw-r--r-- 1 student student     73 Jan 15 13:06  casino1.txt
-rw-r--r-- 1 student student    129 Jan 15 13:00  casino.txt
drwxr-xr-x 2 student student   4096 Nov 14  2024  Desktop
drwxr-xr-x 2 student student   4096 Nov 14  2024  Documents
drwxr-xr-x 2 student student   4096 Nov 14  2024  Downloads
-rw-r--r-- 1 student student     29 Jan 22 12:44  d.txt
drwxr-xr-x 2 student student   4096 Jan 15 13:23  ECM1413_Computers_and_the_Internet
-rw-r--r-- 1 student student    318 Jan 15 13:18  marged_file_backup.txt
-rw-r--r-- 1 student student    318 Jan 15 13:09  merged_file.txt
drwxr-xr-x 2 student student   4096 Nov 14  2024  Music
drwxr-xr-x 2 student student   4096 Nov 14  2024  Pictures
drwxr-xr-x 4 student student   4096 Feb  2 17:49  projects
drwxr-xr-x 6 student student   4096 Nov 25  2024  pt
drwxr-xr-x 2 student student   4096 Nov 14  2024  Public
-rw-r--r-- 1 student student    150 Feb  2 18:18  quotes_copy.txt
-rw-r----- 1 student student    300 Feb  2 18:23  quotes.txt
drwxr-xr-x 2 student student   4096 Nov 14  2024  Templates
drwxr-xr-x 2 student student   4096 Jan 15 13:20  test_dir
-rw-r--r-- 1 student student      0 Jan 22 12:58 '\..text'
drwx------ 1 student student      0 Feb  2 19:46  thinclient_drives
-rw-r--r-- 1 student student 693880 Feb  2 20:09  transcript.txt
-rw-r--r-- 1 student student     76 Jan 22 12:47  t.txt
-rw-r--r-- 1 student student      0 Jan 22 12:58 '\..txt'
drwxr-xr-x 2 student student   4096 Nov 14  2024  Videos
student@lab000001:~$ ls -l quotes.txt
-rw-r----- 1 student student 300 Feb  2 18:23 quotes.txt
student@lab000001:~$
```

Task 9:

```
student@lab000001:~$ tr -d 'aeiouAEIOU' < quotes.txt >> quotes_novowels.txt
student@lab000001:~$ cat quotes_
quotes_copy.txt      quotes_novowels.txt
student@lab000001:~$ cat quotes_novowels.txt
Hll my nm s Cmrn!
 m rlly njyng lnx!
Cmptr Scnc s n mzng sbjct.
 m flng qt cmfrtbl wth th trmnl nw!!!
Hll my nm s Cmrn!
 m rlly njyng lnx!
Cmptr Scnc s n mzng sbjct.
 m flng qt cmfrtbl wth th trmnl nw!!!
student@lab000001:~$
```

Task 10:

```
student@lab000001:~$ tr 'a-z' 'A-Z' < quotes_copy.txt >> quotes.txt
student@lab000001:~$ cat quotes.txt
Hello my name is Cameron!
I am really enjoying linux!
Computer Science is an amazing subject.
I am feeling quite comfortable with the terminal now!!!
Hello my name is Cameron!
I am really enjoying linux!
Computer Science is an amazing subject.
I am feeling quite comfortable with the terminal now!!!
HELLO MY NAME IS CAMERON!
I AM REALLY ENJOYING LINUX!
COMPUTER SCIENCE IS AN AMAZING SUBJECT.
I AM FEELING QUITE COMFORTABLE WITH THE TERMINAL NOW!!!
student@lab000001:~$
```

My Answers to Part 2: Architecture

Task 1: Pipeline Execution Diagram

	CPU Cycles											
Instruction	1	2	3	4	5	6	7	8	9	10	11	12
LW R1, 0(R2)	IF	ID	EX	MEM	WB							
ADD R3, R1, R4		IF	ID	STALL	EX	MEM	WB					
SUB R5, R3, R6			IF	STALL	ID	EX	MEM	WB				
LW R7, 4(R3)					IF	ID	EX	MEM	WB			
AND R8, R7, R1						IF	ID	STALL	EX	MEM	WB	
BEQ R8, R0, LABEL							IF	STALL	ID	EX	MEM	WB
ADD R9, R5, R3									IF	ID	FLUSH	
OR R10, R1, R8										IF	FLUSH	

Note: Instruction 9 would be fetched during cycle 11.

Task 2/3: Hazard Identification/Resolution

Note: I found it easier to identify, explain and then resolve the hazard in one go. This is how I have interpreted questions 2 and 3.

Data Hazards:

1. *RAW Hazard* – Since instruction 1 is a LOAD instruction, the value it reads from memory (at the address held in R2) will be written to register 1 (R1) in the WB stage, and then instruction 2 will need to read this value from R1 in its ID stage in the same cycle (internal forwarding). Fortunately, with hardware forwarding enabled, this value read from memory will be available to instruction 2 at the end of instruction 1's MEM stage, rather than the WB stage. The only way for the CPU to "know" that it must wait for the result from the MEM stage is to decode the ADD instruction in cycle 3 (once fetched in cycle 2). In cycle 4, the CPU must STALL, then the value read from memory is directly forwarded to the input of the ALU for the ADD instruction, meaning that instruction 2's EX stage happens in cycle 5. Since instruction 2 (ADD) was stalled in the ID stage (cycle 4), that means instruction 3 (SUB) must be stalled in the IF stage (also cycle 4). If it were not, it would overwrite instruction 2's data in the ID stage registers, resulting in a structural hazard. Therefore, 1 stall cycle was introduced due to this hazard.
2. *RAW Hazard* – Now, in cycle 5, instruction 3 enters the ID stage, where the CPU realises that instruction 3 needs the result from instruction 2, but it hasn't completed yet. Since we have hardware forwarding enabled and instruction 3 has been stalled in cycle 4, the timing lines up perfectly for forwarding to take place without another stall: instruction 2 directly passes the result from its EX stage into the ALU input for instruction 3's EX stage, no stall required here. Therefore, this hazard didn't introduce anymore stall cycles.

3. *RAW Hazard* – As with data hazard 1, instruction 5 (AND) requires the data from register 7 (R7) that instruction 4 (LW) writes to. Therefore, the CPU identifies this once it has decoded instruction 5 in cycle 7 and then must STALL instruction 5 during cycle 8. Now, at the end of cycle 8, the value instruction 4 has read from memory is ready and is forwarded directly to the EX stage of instruction 5 in cycle 9. Since instruction 5 (AND) has been stalled in the ID stage in cycle 8, instruction 6 (BEQ) must also be stalled in the IF stage in cycle 8. If not, instruction 6 would overwrite instruction 5's data in the ID stage registers, which would cause a structural hazard. Therefore, 1 stall cycle was introduced.
4. *RAW Hazard* – Now that instruction 6 (BEQ) is moved onto the ID stage in cycle 9, the CPU realises that it requires the result of instruction 5 (AND), but it hasn't finished yet. Since instruction 6 has been stalled in cycle 8, the EX stage of instructions 5 and 6 line up perfectly for instruction 5's EX stage result to be forwarded to the ALU inputs for instruction 6's EX stage. Therefore, this data hazard didn't directly introduce another stall cycle, other than the one mentioned above.

Control Hazards:

1. The coursework specification states that branch conditions are evaluated in the EX stage rather than the ID stage.
   In cycle 9, instruction 6 (BEQ) is being decoded. The CPU assumes forward branches (low-level versions of if statements) are always not taken (i.e. don't change the PC's regular PC + 4 operation). This means that while instruction 6 (BEQ) is being decoded, the CPU continues with fetching the subsequent instruction, instruction 7 (ADD), even before it has evaluated the branch condition. Then, in cycle 10, instruction 6 (BEQ) is being executed (evaluated), instruction 7 (ADD) is being decoded, and instruction 8 (OR) is being fetched. At the end of cycle 10, the CPU has evaluated the branch condition and now realises that the branch is taken, meaning that in cycle 11, it must flush the pipeline of instructions 7 and 8, as indicated by the word FLUSH.
   Cycle 9 was the first cycle where the next instruction can start in the pipeline, and the CPU realises instruction 6 is a taken branch in cycle 10; the next useful instruction to enter the pipeline is in cycle 11. This results in 2 penalty/stall cycles being introduced due to the hazard.

Task 4: Performance Analysis

a) Calculate the number of clock cycles required to execute the given code.

Since Task 1 asked us to create the pipeline execution table for instructions 1 through 8, the required CPU cycles are 12. Then, adding in instruction 9 produces the following table:

	CPU Cycles														
Instruction	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
LW R1, 0(R2)	IF	ID	EX	MEM	WB										
ADD R3, R1, R4		IF	ID	STALL	EX	MEM	WB								
SUB R5, R3, R6			IF	STALL	ID	EX	MEM	WB							
LW R7, 4(R3)					IF	ID	EX	MEM	WB						
AND R8, R7, R1						IF	ID	STALL	EX	MEM	WB				
BEQ R8, R0, LABEL							IF	STALL	ID	EX	MEM	WB			
ADD R9, R5, R3									IF	ID	FLUSH				
OR R10, R1, R8										IF	FLUSH				
Label: SUB R11, R9, R4											IF	ID	EX	MEM	WB

Therefore, the full program will complete in **15 CPU cycles**.

b) Compute the CPI (Cycles Per Instruction).

My understanding of this question is that I need to calculate the average CPI by dividing the sum of the number of required CPU cycles for each instruction by the number of instructions.

Instruction	Cycles to Complete
1	5
2	6
3	6
4	5
5	6
6	6
7	-
8	-
9	5

Note: instructions 7 and 8 were flushed, so never completed.

$$\text{Average CPI} = \frac{5+6+6+5+6+6+5}{7} = 5.57142857\ldots = 5.6\ (1\text{ d.p})$$

c) Briefly discuss the impact of hazards on performance.

When a hazard occurs, one of three resolutions takes effect: forwarding, stalling (NOP), or flushing.

When a data hazard occurs, it may be resolved using forwarding; however, there are some scenarios where a stall is inevitable. Such a scenario includes: an arithmetic operation directly succeeding a memory load operation, which depends on the value read from memory. In this case, the result of the LOAD operation's EX stage is only the memory address to read from, so it cannot be forwarded onto the EX stage of the arithmetic operation. The value read from memory is only available, at the earliest, at the end of the LOAD operation's MEM stage, where it is then forwarded onto the EX stage of the arithmetic instruction. Meanwhile, this stall cycle resulted in some parts of the CPU remaining idle, waiting for the value to be read from main memory. This also results in all subsequent instructions being started 1 cycle later.

As alluded to above, when a CPU must perform a stall cycle, parts of the CPU remain idle, not completing any "useful" work. This is time wasted that the CPU will never get back.

A CPU assumes forward branching (if statements) to be not taken and backward branching (loops) to be taken. This means that when a CPU encounters a forward branch operation, before it has even evaluated the condition, it fetches the subsequent instruction from memory (at address PC + 4). If branches are evaluated in the ID stage, which is the stage modern processors evaluate branches in, if the ALU determines that the branch is, in fact, taken, the penalty the CPU must pay is a FLUSH operation. This involves clearing all the IF stage registers of the incorrect instruction, then fetching the correct instruction.

This penalty is magnified if the CPU evaluates branches in the EX stage. In this case, if the branch was taken, the CPU would have to FLUSH two instructions, one from the IF stage and one from the ID stage. This is why modern CPUs are designed to evaluate conditions in the ID stage, meaning that you may require one STALL cycle, but the CPU would only have to FLUSH one instruction.

This is why hazard prevention and dynamic/hybrid branch prediction play a huge role in the design of modern CPUs; eliminating as many hazards as possible will dramatically increase CPU performance.

Task 5: Instruction Reordering

a) New re-organised version of the program:

	Instruction
1	LW R1, 0(R2)
2	ADD R3, R1, R4
3	LW R7, 4(R3)
4	AND R8, R7, R1
5	BEQ R8, R0, LABEL
6	SUB R5, R3, R6
7	ADD R9, R5, R3
8	OR R10, R1, R8
9	Label: SUB R11, R9, R4

Explanation:

Since the only instruction that uses the value of register 5 (R5) is instruction 7 (ADD), it makes sense to move instruction 6 (SUB) from position 3 to position 6, since its result is only required if the branch operation is false (not taken).

As instructions 2 through 5 all rely on the result from their respective previous instruction, they cannot be reordered any further. After instruction 5, instructions 6 and 7 must be completed in the specified order. Instruction 8 can be performed before or after 6 and 7; however, the order doesn't affect the performance.

b) New Pipeline Execution Table for instructions 1 through 8:

	CPU Cycles										
Instruction	1	2	3	4	5	6	7	8	9	10	11
LW R1, 0(R2)	IF	ID	EX	MEM	WB						
ADD R3, R1, R4		IF	ID	STALL	EX	MEM	WB				
LW R7, 4(R3)			IF	STALL	ID	EX	MEM	WB			
AND R8, R7, R1					IF	ID	STALL	EX	MEM	WB	
BEQ R8, R0, LABEL						IF	STALL	ID	EX	MEM	WB
SUB R5, R3, R6								IF	ID	FLUSH	
ADD R9, R5, R3									IF	FLUSH	
OR R10, R1, R8										-	

c) How does my reordering improve performance?

By moving instruction 6 from position 3 to position 6, instructions 3, 4, and 5 start 1 cycle earlier, so instructions 1 through 8 are completed 1 CPU cycle faster.

This can be seen with the original program, where all instructions reached the IF stage; however, in my re-ordered version, the last instruction doesn't even make it to the IF stage.